Palindrome Practice Problems

Oct 16, 2026 · @Kuhu

The idea: expand from the center

Treat every letter as a possible center, then move a `left` and a `right` pointer outward while the two letters match. These problems all reuse that one idea.

```
# Odd length (center is one letter), like "aba"
left = center
right = center

# Even length (center is between two letters), like "abba"
left = center
right = center + 1
```

Problem A: Count all palindromic substrings

Count every palindrome inside the string. Single letters count, and the same palindrome at different positions counts separately.

Input	Output	Palindromes counted
`abc`	3	a, b, c
`aba`	4	a, b, a, aba
`abba`	6	a, b, b, a, bb, abba
`aaa`	6	a, a, a, aa, aa, aaa
`xabay`	6	x, a, b, a, y, aba

Hint: use the same two pointers. Instead of keeping the longest length, add 1 to a counter every time the letters match.

Problem B: Start and end index of the longest palindrome

Find the longest palindromic substring and print its `[start, end]` indices.

Input	Output	Palindrome
`dabac`	`[1, 3]`	aba
`cabbad`	`[1, 4]`	abba
`xyzzyxab`	`[0, 5]`	xyzzyx
`abcd`	`[0, 0]`	any single letter

Hint: whenever you find a new best palindrome, save the current `left` and `right` values.

Problem C: Count centers that make a palindrome of length 3 or more

Use only one-letter centers (the odd case). Count how many centers grow into a palindrome of length 3 or more.

Input	Output	Centers (index)
`abcd`	0	none
`aba`	1	1 (`aba`)
`ababa`	3	1 (`aba`), 2 (`ababa`), 3 (`aba`)
`abcbaxyx`	2	2 (`abcba`), 6 (`xyx`)

Hint: after the loop finishes for a center, check if `right - left - 1` is at least 3.

Problem D: Shortest palindrome by adding letters in front

You may add letters only at the **front** of the string. Return the shortest string that is a palindrome. This is the hardest one, so try it last.

Input	Output	What was added
`aba`	`aba`	nothing
`aacecaaa`	`aaacecaaa`	`a`

Input	Output	What was added
`abb`	`bbabb`	`bb`
`abcd`	`dcbabcd`	`dcb`

Hint: find the longest palindrome that starts at index 0. Reverse the remaining letters and put them in front.

How to practice

1. Start with Problem A. Your code changes by only one line.
2. Move to Problem B, where you learn to save the indices.
3. Write the indices under each letter and dry run from index 0, as in the `dabac` and `cabbad` examples.
4. Compare your answer with the table. If it differs, find the first center where your dry run went off.
5. Try Problems C and D last.